AEGIS-AT: Measuring Attribution Integrity Under Sibling Impersonation in Multi-Agent Delegation

A Reproducible Adversarial Benchmark Showing That RFC 8693 Delegation Can Make Audit Attribution *Worse*, Not Better

Rahul Singh*

June 2026

Abstract

Standards bodies (NIST NCCoE, OpenID Foundation) are actively recommending cryptographic delegation — principally OAuth 2.0 Token Exchange (RFC 8693) — as the primitive that will give multi-agent AI systems *non-repudiation*: the property that every consequential action can be traced to a responsible party. We present **AEGIS-AT**, a controlled, reproducible, adversarial benchmark that tests this assumption directly, and find that it fails in a specific and structural way. AEGIS-AT implements a minimal Security Operations Center (SOC) triage pipeline — two sibling agents sharing one scope-gated tool — and measures an **Attribution Integrity Score** (AIS): the fraction of adversarial actions whose claimed `(actor, scope, principal_chain)` exactly matches an independently observed ground truth. We score AIS across four progressive defense baselines that are *configuration flags over a single codebase*, so the four scores are genuinely comparable rather than apples-to-oranges. The result is a **non-monotonic curve**: attribution is perfect (AIS = 1.0) under simple per-agent identity, then *regresses to zero* (AIS = 0.0) the moment RFC 8693 delegation is added, and tamper-evident logging does not recover it. The cause is not a bug: RFC 8693's "current actor" (`act.sub`) names the agent that *requested* delegated authority, but in a multi-agent hand-off the agent that *executes* the action can differ — and the standard provides no field that records the executor. An attacker who can shape the content of a single security alert can therefore cause a high-consequence action to be taken and attributed to the wrong, lower-privilege sibling, while every cryptographic check passes. The finding is *categorical* (it succeeds by construction, not with some probability) and is verified deterministic: each baseline yields byte-identical records across repeated runs. All AIS values are pre-registered in the threat model before the attack code was written. We argue the gap is structural rather than topology-specific, name the standardized layer hypothesized to close it (sender-constrained tokens, RFC 9449 / RFC 8705), and release the full pipeline (59 tests; reproduces in seconds).

Keywords: multi-agent systems, AI agent security, attribution, non-repudiation, OAuth 2.0 Token Exchange, RFC 8693, confused deputy, delegation, audit logging, benchmark.

Contents

*Independent researcher, Jersey City, NJ. Correspondence: rahul.rs1397@gmail.com · github.com/rahulsingh1397.

1 Introduction

Multi-agent AI systems increasingly take consequential, irreversible actions: quarantining hosts, blocking accounts, modifying infrastructure. When such an action is taken, an audit log is expected to answer one question: *which agent did this?* The dominant answer being standardized is cryptographic delegation. A human principal authenticates once; an orchestrator mints scoped delegation tokens for each agent; the token carries a verifiable chain of "who acts on whose behalf." OAuth 2.0 Token Exchange (RFC 8693) is the mechanism, and NIST's NCCoE and the OpenID Foundation have both, in early 2026, named auditing and non-repudiation of AI agent actions as an open problem to be solved with exactly these primitives [4, 5].

This paper asks whether that assumption survives contact with a realistic adversary, and answers: *not in the multi-agent re-delegation case.* We build the smallest system in which the failure can occur — two sibling agents (`Agent-Enrich`, read-only; `Agent-Contain`, write-capable) sharing one scope-gated tool — and we measure attribution correctness under a sibling-impersonation attack across four defense configurations.

The finding. Attribution is *perfect* under simple per-agent identity (Baseline 2, AIS = 1.0). Adding RFC 8693 delegation (Baseline 3) drives it to AIS = 0.0, and tamper-evident logging (Baseline 4) leaves it there. The curve is non-monotonic: the two primitives most emphasized for agent non-repudiation — signed delegation chains and tamper-evident logs — do not close the gap, and signed delegation *actively opens* it relative to plain per-agent identity. Table 1 states the result.

Table 1: The measured Attribution Integrity Score across four baselines. Each baseline is a configuration flag over one codebase; only the credential differs. Measured values reproduce the pre-registered prediction exactly.

Baseline	Defense in place	Signal read	Tracks executor?	ais
B1	Shared service account	shared credential	undefined	**0.0**
B2	Per-agent identity	execution-time authenticator	yes	**1.0**
B3	+ RFC 8693 delegation	delegation current actor	no	**0.0**
B4	+ tamper-evident log	delegation current actor	no	**0.0**

Why it regresses (a one-paragraph preview). RFC 8693's "current actor" (`act.sub`) is the party that *requested* the delegated authority. In a multi-agent hand-off, the agent that *executes* the action — the one that actually presents the token at the resource — can differ from the one named in the token. The standard provides no field that records the executor when it differs from the requester. RFC 8693 §4.1's normative `MUST` is scoped to the *access-control decision* ("consider only the current actor"), not to audit logging; a realistic implementation reuses that same access-control identity for the audit record. Combined with unbound bearer tokens (RFC 8693 inherits OAuth 2.0's default holder model, which does not require sender-constraint) and a topology where minting precedes execution-routing, the system logs the *requester*. The standard neither prevents nor mandates this; it simply has no place to put the executor.

Contributions.

1. **A metric.** The Attribution Integrity Score (AIS): a strict, three-field (`actor`, `scope`, `principal_chain`) measure of whether a claimed audit record matches an independently observed ground truth,

with a defect breakdown that diagnoses *which* field broke (§4).

2. **A benchmark.** Four defense baselines realized as configuration flags over a single codebase, so AIS differences are attributable to the flag and not to incidental implementation quality (§6).
3. **An independent ground-truth construction.** A harness recorder that observes the true executing agent out-of-band — from the execution context, never from the token — making the comparison non-circular (§7.1).
4. **A pre-registered, reproduced result.** The non-monotonic curve, predicted in the threat model before the attack code was written, reproduced deterministically against the real modules; the attack succeeds *by construction* (§9).
5. **A structural argument and a named fix.** An analysis of why the gap is topology-independent, and identification of the standardized layer (sender-constrained tokens) hypothesized to close it, scoped as future work (§10, §12).

Scope and honesty up front. This is one minimal system ($n = 1$), with deterministic scripted agents (no LLM), and Baseline 4 is attribution-only in v1 (the hash-chained tamper-evident log is named as future work). These are deliberate boundaries that buy a clean causal claim; we state them here and defend them in §11 rather than let a reviewer find them.

2 Background and Related Work

2.1 RFC 8693 delegation and the `act` claim

OAuth 2.0 Token Exchange (RFC 8693) represents delegation with the `act` ("actor") claim. A token's `sub` names the principal whose authority is being exercised; the `act` claim names who is acting on that principal's behalf, and `act` can nest, forming a verifiable chain back to the original human. RFC 8693 §4.1 is explicit that, for access-control purposes, a consumer `MUST` consider only the token's top-level claims and the party identified as the *current actor* by the (outermost) `act` claim; prior actors in nested `act` claims are "informational only." The spec's own delegation example (§A.2.5) and its description of the actor as "the actor that will wield the security token" (§A.2.3) implicitly assume that the party who *requested* the token and the party who *wields* it at the resource are the same entity. That assumption is the seam this paper pries open.

2.2 The confused deputy, old and new

The confused-deputy pattern — a privileged component induced to misuse its authority on behalf of a less-privileged party — dates to Hardy [11]. In agentic AI it has re-emerged as a high-severity pattern: the Cloud Security Alliance's March 2026 research note [6] establishes confused-deputy prompt injection as a high-severity threat in agent deployments and observes that when an action runs under a trusted agent's identity, "audit logs may look legitimate and delay detection" — precisely the attribution failure we measure. The most public production instance is *Clinejection* (February 2026): a crafted GitHub issue title drove an AI triage agent into a supply-chain compromise, an unauthorized npm package reaching ∼4,000 developer machines in an eight-hour window [8]. The Salesloft Drift breach (August 2025) [9] is a production demonstration of the *unbound bearer-token* weakness our Baseline 3 depends on: stolen OAuth tokens, presented by a party that was not their legitimate holder, were used to exfiltrate data from 700+ organizations.

2.3 The standards call

NIST's NCCoE concept paper *Accelerating the Adoption of Software and AI Agent Identity and Authorization* (February 2026) [4] names auditing and non-repudiation of AI agent actions as an open problem and asks how existing identity standards (OAuth 2.0, OpenID, SPIFFE) should apply to multi-agent delegation, including the unresolved multi-hop case. The OpenID Foundation's response [5] frames the most urgent risks as failures of *trust* — "Who authorised this agent to act? On whose behalf? Can that be verified?" — and notes that today's deployments rely on unsigned credentials and no clear chain of accountability. The Foundation for American Innovation's *Human-Anchored Intent-Bound Delegation* (HAID) proposal [7] offers a candidate fix: signed, scope-attenuating, human-anchored delegation chains — the kind of execution-identity binding we name as future work.

2.4 Closest prior academic work

The Misattribution Gap [10] measures misattribution in agentic systems across 64 documented failures, finding that attribution systems consistently blamed the model rather than the poisoned memory layer. It is adjacent but a *different layer*: it measures model-vs-memory misattribution (memory poisoning, semantic norm drift), whereas AEGIS-AT measures agent-vs-agent sibling impersonation at the delegation layer. To our knowledge no published adversarial benchmark scores attribution integrity for the sibling-impersonation case across a structured defense gradient; the problem space is *defensibly underexplored* rather than untouched.

3 Threat Model

3.1 System under test

AEGIS-AT models a minimal SOC alert-triage pipeline (Figure 1). A human analyst authenticates and issues a request to a triage orchestrator. The orchestrator delegates to two sibling subagents by minting each a scoped RFC 8693 delegation token. Both subagents can call one shared tool, `siem_action`, which records the identity of the calling agent in an action log.

- **Human principal** (`human:analyst`) — authenticates once, originates the task, is the root of every delegation chain; does not call tools directly.
- **Orchestrator** — a stateless RFC 8693 token-exchange endpoint. It validates an exchange request and mints the appropriately scoped, correctly nested token. It does *not* read alert content to make routing decisions.
- **Agent-Enrich** (Subagent A) — the lower-consequence sibling; read-only context gathering under `siem:read`. In the attack, this is the innocent sibling whose identity is falsely stamped on Contain's action.
- **Agent-Contain** (Subagent B) — the higher-consequence sibling and the true executor; consequential response actions under `siem:write`.
- **Tool** `siem_action` — a single SOAR-style endpoint, scope-gated. The `command` parameter selects the operation; the token's `scope` claim decides whether the call is permitted. The tool reads agent identity *only* from the verified `act` claim, never a self-reported field.

This is deliberately the smallest configuration in which sibling impersonation is possible: two siblings (so impersonation has a target) and one shared tool (so the attack has exactly one degree

of freedom — A acted, the log says B). The privilege asymmetry between Contain and Enrich is what makes misattribution *security-relevant* rather than cosmetic.

Step / actor	What happens (and at which boundary)	
`human:analyst`	issues the triage request	*(Boundary 1)*
↓	orchestrator mints scoped RFC 8693 tokens	*(Boundary 2 — attack)*
`Agent-Enrich`	(requester, `siem:read`) sends a re-delegation request	
↓	orchestrator mints a `siem:write` token *naming Enrich*	
`Agent-Contain`	(true executor) wields the token, calls `siem_action` *(Boundary 3: all checks pass)*	
↓	tool writes `claimed_actor = enrich`	*(Boundary 4)*
`Harness recorder`	observes `true_actor = contain`, out-of-band	*(Boundary 5)*
AIS scorer	`claimed` $\neq$ `true` $\Rightarrow$ attribution defect	

Figure 1: The AEGIS-AT pipeline and its five trust boundaries. The attack lives at Boundary 2 (token minting); Boundaries 3 and 4 faithfully transcribe a record that is already wrong; Boundary 5 is the independent measurement instrument.

3.2 Trust boundaries

The system has five boundaries; naming all five lets a reviewer see that the attack lives at exactly one of them.

Boundary 1 (principal → orchestrator).
The analyst authenticates via OIDC; their task prompt is trusted. *But* the pipeline also ingests upstream SIEM alert content, which is definitionally untrusted — it describes activity outside the perimeter and routinely carries attacker-controlled strings. This is the attacker's sole injection point.

Boundary 2 (orchestrator → subagent — minting).
The attack boundary. Cryptographic verification is sound (the signing key is out of reach), but the *decisional* content of the exchange request is shaped by upstream, attacker-influenceable input. The orchestrator honestly mints a token that names the wrong sibling as actor.

Boundary 3 (subagent → tool).
The tool runs full verification — signature, expiry, chain integrity, scope gate, identity resolution — and *every check passes*. It then faithfully records the current actor from the verified `act` claim. The misattribution is not a Boundary 3 failure; it is a Boundary 2 outcome that Boundary 3 honestly transcribes.

Boundary 4 (tool → action log).
The claimed actor is written to the log. The wrong actor arrived already wrong. In Baselines 1–3 the log is a plain append-only file; Baseline 4's tamper-evident design (future work in v1) protects the integrity of the (wrong) record but cannot detect the upstream misattribution.

Boundary 5 (system → harness — ground-truth recorder).
The measurement instrument, not a defense. It observes the true executing agent out-of-band, independent of any token or claim.

3.3 Adversary model

The adversary is modeled at the *gray-box* level: they know the kind of system and the kind of decisions it makes, but not its exact prompts, extraction code, or scope strings. This matches every real-world precedent cited above.

- **Can:** control the text of one or more SIEM alerts processed by Enrich (hostnames, URLs, log lines, etc.); cause those alerts to enter through normal channels; repeat the attempt (no rate limit assumed).
- **Cannot:** compromise the analyst, SIEM, or upstream systems; forge or modify tokens (the signing key is out of reach); read or write the ground-truth recorder; compromise the orchestrator, subagents, or tool *as processes* — only the data they consume.

Goal: cause an action truly performed by Agent-Contain to be attributed to Agent-Enrich, so that an analyst reviewing the log sees no record of the high-consequence agent having taken the high-consequence action. The attack is "covering tracks," not privilege escalation: the attacker hides *who* exercised authority that already legitimately existed. Critically, the attack needs *no* zero-day, no jailbreak, no instruction-override, no adversarial embedding — only that a genuinely containment-warranting alert flow through the re-delegation path.

4 The Attribution Integrity Score

4.1 Ground-truth and claimed schemas

For each tool invocation the harness's ground-truth recorder writes a tuple

$$\langle \texttt{true\_actor}, \texttt{true\_scope}, \texttt{true\_principal\_chain}, \texttt{command}, \texttt{target}, \texttt{timestamp} \rangle,$$

where `true_actor` is the agent that actually executed the call (observed from the executing context, not any token); `true_scope` is the scope the command genuinely requires (a static command→scope map); and `true_principal_chain` is the ordered delegation path from immediate actor to human principal, [`true_actor`, `human:analyst`] — a two-hop chain. The orchestrator does *not* appear in it: it is a stateless minting endpoint, not a delegated principal holding a token, so RFC 8693's `act` claim records no hop for it.[1]

The corresponding Boundary 4 record has the parallel shape with each `claimed_*` field derived from the verified `act` claim of the presented token. Records are matched between the two stores by the (`command`, `target`, `timestamp`) triple.

4.2 The metric

For a single adversarial action a, define

$$\mathsf{is\_correct}(a) = \begin{cases} 1 & \text{if } \texttt{claimed\_actor}(a) = \texttt{true\_actor}(a) \\ & \quad \wedge\ \texttt{claimed\_scope}(a) = \texttt{true\_scope}(a) \\ & \quad \wedge\ \texttt{claimed\_principal\_chain}(a) = \texttt{true\_principal\_chain}(a) \\ 0 & \text{otherwise.} \end{cases}$$

[1] This two-hop shape is itself a finding in miniature. An earlier draft of the threat model recorded a three-hop chain [`actor`, `orchestrator`, `analyst`] — the natural practitioner intuition. That intuition is wrong about RFC 8693, and the gap between how engineers reason about delegation chains and what the standard actually records is exactly the gap that makes this misattribution surface.

Comparison is strict: all three fields must match. `principal_chain` comparison is ordered-list equality (a permutation, missing hop, or inserted hop is a defect); where no delegation chain exists (opaque per-agent credentials at Baselines 1–2), the chain is `None` and matches `None`. The Attribution Integrity Score for a baseline B is

$$\text{AIS}(B) = \frac{1}{|A(B)|} \sum_{a \in A(B)} \mathsf{is\_correct}(a),$$

where $A(B)$ is the set of adversarial actions executed under B — tool calls the attack actually influenced. Non-adversarial calls (setup, calibration) are excluded from the denominator. AIS is reported *per baseline*; the curve across baselines is the result.

4.3 Derived reporting

Two diagnostics travel with each AIS. The **defect breakdown** records, for each incorrect action, which of the three fields mismatched — showing whether a baseline breaks attribution uniformly or concentrates on one field. The **hold rate** reports the marginal AIS change at each baseline transition (1→2, 2→3, 3→4), isolating each defense layer's contribution.

4.4 Why the v1 measurement is categorical

The v1 attack is *categorical, not stochastic*: under scripted deterministic agents the misattribution succeeds by construction on every adversarial action. The finding is therefore a curve *shape*, not a frequency estimate. Confidence intervals are degenerate by design (AIS $\in \{0, 1\}$ per baseline). The harness establishes this with a determinism check that proves each baseline yields byte-identical records across repeated runs, so a single canonical execution per baseline suffices. A probabilistic-policy sweep that would make AIS a real attack-frequency estimate (with Wilson intervals) is named as future work.

5 The Attack Mechanism

5.1 The structural property exploited

Under RFC 8693, when agent X requests a delegated token to act on behalf of principal P, the resulting token's `act` claim records X as the current actor. This is correct: the chain answers "on whose authority, through which delegating parties." But an audit log is trusted to answer a subtly different question — "who *performed* this action?" In a single-agent setting the two coincide. In a multi-agent setting where one agent requests a delegated capability that *another* agent executes, they diverge, and the divergence is invisible to every cryptographic check because nothing was forged or malformed.

5.2 Token structure under Baselines 3–4

When the orchestrator performs the re-delegation exchange, the issued token follows RFC 8693 §4.1 delegation semantics:

```
sub:   "human:analyst"        <- principal (on whose behalf)
scope: "siem:write"
act: {
  sub: "agent:enrich"         <- current actor (requester / named wielder)
}
```

The chain is two hops: current actor (`agent:enrich`) and root principal (`human:analyst`). Two properties are decisive, both following directly from §4.1: (i) the current actor is the *requester*, not the executor; and (ii) the executing agent (`agent:contain`) *does not appear in the token at all*. There is no field in the spec-compliant structure that records "who wielded the token at the resource" distinct from "who was delegated the authority." The misattribution is therefore not a property of any field being read incorrectly; it is a property of the token having no field that can express the divergence.

5.3 The attack, step by step

The attacker's only capability is to control the content of an alert Enrich processes. The attack does not require fooling Enrich, embedding instructions, or causing any component to misbehave — only that a genuinely containment-warranting alert flow through the Enrich → re-delegation path.

1. The alert legitimately contains containment-warranting indicators (the attacker shaped them, but they are *real* indicators). Enrich is not deceived; it makes the right call.
2. Enrich correctly initiates a re-delegation request for a containment action, presenting its own token as the `actor_token`.
3. The orchestrator correctly mints a `siem:write` token whose `act` chain names Enrich (the requester).
4. Agent-Contain — the agent that actually holds and runs the containment capability — wields the token and calls `siem_action`.
5. The tool verifies the token (every Boundary 3 check passes), reads the current actor (`enrich`), and records `claimed_actor` = `enrich`.
6. The ground-truth recorder observes `true_actor` = `contain`.
7. The scorer flags an actor mismatch: claimed Enrich, true Contain. Attribution has failed.

An analyst reviewing the log sees the high-consequence containment attributed to the read-only enrichment agent; the true executor appears nowhere in the record.

5.4 Why it survives every objection

"The orchestrator has a bug." No — it validated cryptographically and built the chain from the presented `actor_token`, exactly as §4.1 specifies. **"Enrich was prompt-injected."** No — Enrich made the correct decision; the result holds even if Enrich is a perfect, unfoolable agent. **"The tool skipped a check."** No — every check passed; the tool faithfully recorded the current actor. **"You didn't sanitize alert text."** No — no component read identity from alert text; alert content's only role was to legitimately trigger a containment-warranting situation. The attack works because the delegation chain answers "who requested" while the audit log is trusted to answer "who acted," and in the re-delegation pattern those are different agents.

6 Defense Baselines

The benchmark measures AIS across four configurations, applied as config flags over one codebase (not four implementations). Each adds one layer to the previous; *only the credential differs*. For each baseline we state the signal the tool reads and whether it tracks the true executor.

Baseline 1 — Shared service account.
All agents share one credential. The tool cannot distinguish callers; attribution is *undefined* (not wrong about a specific sibling — there is no per-agent identity to be right about). Predicted AIS ≈ 0.0. This is the common-but-wrong status quo (shared API keys).

Baseline 2 — Per-agent identity.
Each agent holds its own credential; attribution is bound at authentication time. Contain executes, so Contain's credential authenticates, so the tool records Contain. *Claimed actor = true actor.* Predicted AIS ≈ 1.0. Attribution is correct not by design sophistication but because here the *executor is the authenticator.*

Baseline 3 — + RFC 8693 delegation.
The tool now resolves the claimed actor from the delegation chain's current actor (`act.sub` = Enrich, the requester), as §4.1 mandates for the access-control decision. Under unbound bearer tokens, Contain lifts the token minted for Enrich and presents it, and nothing in the protocol detects the substitution. Predicted AIS ≈ 0.0. This is the central result: signed delegation *regresses* attribution relative to per-agent identity, by following the standard correctly.

Baseline 4 — + tamper-evident log.
Same signal as B3. Tamper-evidence protects the integrity of the recorded entry but does not change *what* is recorded; the wrong actor was committed upstream at mint time. Predicted AIS ≈ 0.0, unchanged from B3. (In v1 this module is not built — B4's attribution equals B3 by construction; a real hash-chained log, which tests log *integrity*, a separate metric, is future work.)

The two ingredients of the regression are neither sufficient alone: (i) unbound bearer tokens carrying a current-actor claim (RFC 8693 inherits OAuth 2.0's default holder model and does not require sender-constraint), and (ii) a multi-agent hand-off where the wielder differs from the issuer-named actor (the orchestrator must name the requester because the executor is undetermined at mint time). §4.1's `MUST` is scoped to access control and is silent on audit; a realistic implementation reuses the access-control identity for the audit record, and that identity is necessarily the named requester.

7 Implementation

The system is implemented in Python (∼6 core modules), using PyJWT for RS256-signed tokens and the `cryptography` library for key generation. The agents are scripted (deterministic) by design — this isolates the delegation-layer failure from model behavior.

`auth/tokens.py`
Mints and verifies RFC 8693 tokens by hand: `mint_initial_token`, `exchange_token` (nests `act`, narrows scope, refuses to widen), `verify_token`, and `actor_chain`, which returns the path *current-actor-first, root-principal-last.* Identity resolution reads `chain[0]` — the current actor (§4.1), never the innermost subject (the root principal).

`policy/scope_map.py`
The shared command→scope contract (`keyword_search` → `siem:read`, `isolate_host` → `siem:write`). Imported by both the tool and the recorder so they cannot drift. Unknown commands raise — a fail-loud policy error, not a default.

`tools/siem_action.py`
: The scope-gated tool (Boundary 3): signature + expiry (PyJWT), chain integrity, scope gate, identity resolution. For opaque per-agent credentials (Baselines 1–2) it reads the authenticating agent and sets `principal_chain = None`; for JWTs (Baselines 3–4) it takes the `act`-claim path. Discrimination is on credential *structure* only.

`orchestrator/orchestrator.py`
: A thin RFC 8693 validator/minter (Boundary 2). The only validation it adds beyond `exchange_token` is that the `sub` must name a human principal. It does not route on or read alert content; it does not appear in the minted chain.

`harness/recorder.py`
: The independent ground-truth recorder (Boundary 5); see §7.1.

`harness/scorer.py`
: The AIS metric, defect breakdown, and Wilson CI; plus `is_non_monotonic(curve)`, the named predicate that pins the §6 headline claim in one place.

`harness/sweep.py`
: The baseline switch: `run(baseline)` mints the per-baseline credential (the only thing that varies), runs the canonical attack once on a thread named for the true executor, and scores it; `verify_deterministic` and `emit_curve` compose it.

7.1 The independent ground-truth recorder

The recorder wraps `siem_action` and observes the calling thread's name as `true_actor` *before* forwarding the call. Its independence rests on three axes: a **process boundary** (it runs in the harness, distinct from the agent contexts the attacker influences); **credential isolation** (no in-system component holds ground-truth write credentials); and **causal precedence** (it records before the tool's verification logic runs). The attacker's only capability — alert-content control — cannot cross any of these. If this independence failed, the measurement would be circular; it is established by construction. A v1 honesty note: the recorder uses Python's `threading.current_thread().name` as a proxy for OS process identity; a true process boundary via `multiprocessing`/`os.getpid()` is v2 hardening. Within the threat model (the adversary controls alert text only, not agent code), the proxy holds.

8 Experimental Methodology

Pre-registration. Every AIS value (B1 ≈ 0, B2 ≈ 1.0, B3 ≈ 0, B4 ≈ 0) was committed in the threat model *before* the attack code was written. The test suite asserts the measured curve against these predictions; a contradicted prediction is to be reported as a finding, not reconciled. This is the structural difference between a benchmark and a demo.

Determinism as a checked property. `verify_deterministic(baseline, k)` runs each baseline $k = 5$ times under a fixed clock and asserts byte-identical `(claimed, truth)` records, raising on the first divergent field. `emit_curve` gates on this check before emitting any AIS value, so the curve is only produced once determinism is proven rather than asserted. A fault-injection test feeds a deliberately drifting clock to confirm the check actually catches divergence.

Comparability by construction. Because the four baselines are config flags over one codebase — identical tool, recorder, and scorer code, differing only in the credential — an AIS difference is attributable to the credential flag, not to incidental implementation quality. This is what makes the four numbers a curve rather than four anecdotes.

Reproduction.

```
pip install -r aegis-at/requirements.txt
pytest tests/core -v          # 59 tests
bash scripts/check.sh         # lint, format, invariant greps, test gate
```

The full pipeline reproduces in seconds.

9 Results

9.1 The curve

The measured curve reproduces the pre-registered prediction exactly (Table 2). `is_non_monotonic(curve)` returns `True`: $B2 > B1$, $B2 > B3$, and $B4 = B3$ (byte-identical, since B4 runs B3's code path in v1).

Table 2: Measured AIS with defect breakdown. Numerator/denominator are over the single canonical adversarial action per baseline. The defect breakdown names the claimed vs. true actor and which fields mismatched.

B	Defense	ais	num/den	Predicted	Defect (claimed → true)
B1	Shared account	0.0	0/1	≈ 0	actor: `svc:soar` → `agent:contain`
B2	Per-agent identity	1.0	1/1	≈ 1.0	— (perfect)
B3	+ RFC 8693 deleg.	0.0	0/1	≈ 0	actor + chain: `agent:enrich` → `agent:contain`
B4	+ tamper-evident	0.0	0/1	≈ 0	actor + chain: `agent:enrich` → `agent:contain`

9.2 Reading the defect breakdown

The defect breakdown is the cleanest evidence of the mechanism. At B3 and B4 the system claims `agent:enrich` (the requester named in `act.sub`) while the true executor is `agent:contain`. The defect flags the `actor` *and* `principal_chain` fields together — never `scope`. This confirms a property predicted in the threat model: the actor and principal-chain defects are *correlated*, because both flag precisely when Enrich occupies the current-actor position. The correlation is a true property of the attack, not a metric artifact. At B1 the single defect is on `actor` alone (`svc:soar` vs. `contain`), with `principal_chain` = `None` on both sides (no chain pre-delegation) — the "undefined attribution" case.

9.3 Test and gate status

The full suite is **59 tests, all passing**; the mechanical gate (`scripts/check.sh`) exits 0, enforcing the greppable invariants (the tool is named `siem_action` everywhere; identity resolves to the most-recent actor, never the "innermost" subject), lint (ruff), and format (black). The auth smoke test demonstrates chain nesting, scope narrowing, and forgery rejection. B4 equals B3 byte-for-byte, confirming that tamper-evidence is orthogonal to attribution: the wrong actor is committed before any logging layer sees the entry.

10 Discussion

This is not a logic bug a careful engineer would fix. The honest defense is topological, not "the spec forces it." The fix a reviewer reaches for is "name the wielder, not the requester." But in the realistic topology, the orchestrator mints the delegated token and hands it downstream, where routing decides which sibling executes; the wielder is *not determined at mint time*, so the orchestrator cannot place it in the current-actor claim. Naming the requester is what any orchestrator must do when minting precedes execution-routing. Closing the gap requires binding identity at execution time — the wielder re-exchanges the token on receipt, naming itself current actor, or sender-constrained tokens prevent Contain from presenting Enrich's token at all. Both are exactly the execution-identity binding we name as future work (§12). The objection does not dismiss the result; it identifies the missing layer the result measures the absence of.

The gap is structural, not merely adversarial. The misattribution is a *latent property of the re-delegation pattern*: any containment action re-delegated through Enrich produces the same wrong-actor record, attack or no attack. The attacker does not create the vulnerability; the attacker makes a pre-existing weakness profitable by triggering it deliberately and repeatedly. v1 scopes the AIS denominator to attacker-triggered actions for a clean measurement; an expanded denominator over all re-delegated containment actions (v2) would measure the structural property more faithfully.

Tamper-evident logging protects a wrong answer. Baseline 4 underperforms a naive expectation, and this is the point: the second primitive standards bodies emphasize for non-repudiation cannot recover correct attribution, because the misattribution is established *before* logging. It cryptographically preserves the wrong record. Stating this before the result forecloses the reading "your tamper-evident log is broken" — it is working correctly; it is simply not the right defense for this attack class. That negative result is part of the contribution.

11 Limitations and Validity Threats

We list the ways the result could be wrong or unconvincing and the mitigation for each. The first is a genuine limitation we concede rather than defeat.

1. **Generalization ($n = 1$) — conceded.** The curve is demonstrated on one minimal pipeline with one orchestrator design and one re-delegation topology. v1 establishes the gap *exists*, is triggerable by a realistic gray-box adversary, and survives the two emphasized defenses — *in this system*. What makes it meaningful despite $n = 1$ is *why* it appears: it follows from §4.1 scoping its `MUST` to access control (reused for audit) plus the standard's implicit assumption that the current actor and executor coincide. That assumption is topology-independent; wherever a multi-agent system separates the requester of a capability from its executor, the gap should appear. Establishing prevalence across real architectures is the primary item of future work.
2. **"The system is a toy."** Minimal by design, but every structural element maps to a documented real-world pattern: alert-as-injection-vector (Clinejection, Log4Shell, Splunk XSS), data-driven SOAR routing, and RFC 8693 delegation as the recommended non-repudiation primitive. Minimality isolates the one degree of freedom (requester $\neq$ executor) without confounds.
3. **"Ground truth isn't independent of the log."** Independence is established by construction along three axes (§7.1); the attacker's only capability cannot cross any of them. Honest sub-limitation: established by argument, not formal verification.

4. **"Baselines aren't a fair comparison."** They are config flags over one codebase; differences cannot be attributed to incidental quality. The most attackable point — B2 reaching ≈ 1.0 — rests on a stated model (attribution binds to the execution-time authenticator) with the authorization-vs-attribution distinction made explicit.

5. **"Scripted, not real LLM behavior."** Deliberate: the attack is designed to be independent of agent reasoning quality (Enrich's escalation is the *correct* response to a genuinely warranting alert), removing model-specific confounds. It also means v1 does not measure interaction with imperfect agent decisions — named out of scope.

6. **Thread proxy for process identity.** The recorder uses `threading.current_thread().name` as a v1 proxy; a renamed thread could spoof ground truth, excluded by the adversary model (alert text only). v2 uses `multiprocessing/os.getpid()`.

7. **Baseline 4 is attribution-only in v1.** B4's attribution equals B3 by construction; a real hash-chained tamper-evident log (testing log *integrity*, a separate metric) is future work.

12 Future Work

- **Baseline 5 — sender-constrained tokens.** The gap requires unbound bearer tokens. DPoP (RFC 9449) and mutual-TLS-bound tokens (RFC 8705) bind a token to its holder; under either, Contain cannot present Enrich's token and must obtain its own, so the current actor would track the executor. Whether this recovers the curve is the primary defensive item of future work — effectively a Baseline 5.
- **Prevalence across topologies.** Reproduce the mechanism in additional multi-agent architectures to move from "shown in one instance" to "prevalent across deployments."
- **Stochastic policy / expanded denominator.** A probabilistic agent policy under which AIS becomes a real attack-frequency estimate (Wilson intervals, $N \geq 100$ per baseline), and a denominator over all re-delegated containment actions to measure the latent structural property directly.
- **Tamper-evident log integrity.** Implement the hash-chained, signed log to measure log *integrity* as a metric distinct from attribution.
- **Other sibling-impersonation variants.** Delegation forgery, scope-attenuation bypass, audit-log tampering, principal laundering — each a distinct mode, backlogged for v2.

13 Conclusion

Adding the industry-standard delegation mechanism to a correctly-functioning multi-agent system can make audit attribution *worse*, not better. AEGIS-AT measures this as a non-monotonic Attribution Integrity curve: perfect under per-agent identity, zero once RFC 8693 delegation is added, still zero with tamper-evident logging. The cause is structural — RFC 8693's current actor names the requester, the executor can differ, and the standard has no field for the executor — so the result is not a bug to be patched but a missing layer to be added: execution-identity binding via sender-constrained tokens. The benchmark is small, deterministic, pre-registered, and reproducible; its value is in the trustworthiness of the measurement, not the breadth of coverage. As standards bodies move to make RFC 8693 the backbone of AI agent non-repudiation, AEGIS-AT is a concrete,

falsifiable warning that delegation alone does not buy accountability in the multi-agent case — and a precise statement of what does.

Reproducibility and Artifact Availability

The complete benchmark — threat model, six core modules, harness, and 59 tests — is released. Code (`aegis-at/`, `tests/`, `scripts/`) is licensed Apache-2.0; documentation (`Documents/`) is licensed CC BY 4.0. Every AIS value is asserted in the test suite against the curve predicted in the threat model before the attack code was written. The pipeline reproduces in seconds via `pytest tests/core` and `bash scripts/check.sh`.

References

[1] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore, *OAuth 2.0 Token Exchange*, RFC 8693, IETF, January 2020. https://www.rfc-editor.org/rfc/rfc8693.

[2] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, *OAuth 2.0 Demonstrating Proof of Possession (DPoP)*, RFC 9449, IETF, September 2023. https://www.rfc-editor.org/rfc/rfc9449.

[3] B. Campbell, J. Bradley, N. Sakimura, and T. Lodderstedt, *OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens*, RFC 8705, IETF, February 2020. https://www.rfc-editor.org/rfc/rfc8705.

[4] H. Booth, W. Fisher, R. Galluzzo, and J. Roberts, *Accelerating the Adoption of Software and Artificial Intelligence Agent Identity and Authorization*, NIST NCCoE Concept Paper (Initial Public Draft), February 5, 2026. https://www.nccoe.nist.gov/publications/other/accelerating-adoption-software-and-ai-agent-identity-and-authorization-concept.

[5] OpenID Foundation, *OIDF Responds to NIST on AI Agent Security*, March 11, 2026. https://openid.net/oidf-responds-to-nist-on-ai-agent-security/.

[6] Cloud Security Alliance AI Safety Initiative, *Confused Deputy Attacks on Autonomous AI Agents* (Research Note), March 23, 2026. https://labs.cloudsecurityalliance.org/research/csa-research-note-ai-agent-confused-deputy-prompt-injection/.

[7] Foundation for American Innovation, *Human-Anchored Intent-Bound Delegation (HAID) for AI Agents*, submitted to NIST, April 2, 2026. https://www.thefai.org/posts/human-anchored-intent-bound-delegation-for-ai-agents.

[8] Snyk, *How "Clinejection" Turned an AI Bot into a Supply Chain Attack*, February 2026. https://snyk.io/blog/cline-supply-chain-attack-prompt-injection-github-actions/.

[9] Google Threat Intelligence Group, *Data Theft from Salesforce Instances via the Salesloft Drift Integration (UNC6395)*, August 2025. https://cloud.google.com/blog/topics/threat-intelligence/data-theft-salesforce-instances-via-salesloft-drift.

[10] T. Ahad, I. Hossain, M. J. Alam, S. Puppala, S. B. Alam, and S. Talukder, *The Misattribution Gap: When Memory Poisoning Looks Like Model Failure in Agentic AI Systems*, arXiv:2605.22842, May 2026. https://arxiv.org/abs/2605.22842.

[11] N. Hardy, *The Confused Deputy (or why capabilities might have been invented)*, ACM SIGOPS Operating Systems Review, 22(4), 1988.